

CS 121 Search Engine

Milestone 3 Report

Kevin Guo

June 3, 2026

1 System Overview

This project implements a search engine for the full ICS developer corpus without using a text-indexing library. It consists of two separate Python programs:

- `indexer.py` parses the crawled JSON documents and builds a disk-based inverted index.
- `searcher.py` accepts console queries, retrieves postings from disk, calculates TF-IDF-based scores, and returns ranked URLs.

The Milestone 3 retrieval component uses ranked retrieval rather than Boolean AND retrieval. A document containing any available query term can receive a score. Documents matching multiple query terms accumulate more TF-IDF score and receive an additional coverage benefit, but a document is not excluded merely because it is missing one query term.

2 Indexer Design

2.1 Document Processing

Each corpus document is read from JSON using its `url` and `content` fields. URL fragments are removed before the URL is stored. HTML is parsed with BeautifulSoup, and script, style, and noscript content is excluded. Plain-text documents and unusually large HTML documents use a lighter extraction path to avoid pathological parsing time while still indexing their textual content.

Tokens are all alphanumeric sequences matched by `[A-Za-z0-9]+`. Tokens are lowercased and Porter stemmed. Stop words and numeric tokens are retained to follow the no-stopping and all-alphanumeric-token requirements.

2.2 Important Words

Text appearing in `title`, `h1`, `h2`, `h3`, `b`, and `strong` tags is counted separately. Each posting stores:

(document ID, term frequency, important-term frequency)

The searcher applies an important-word multiplier during scoring so occurrences in these tags contribute more strongly than ordinary body text.

2.3 Partial Indexes and Merge

The in-memory inverted index is flushed to a sorted partial index every 5,000 documents. On the full corpus this produces more than the required three offloads. After indexing, a k-way heap merge combines the partial indexes into one sorted `index.txt` file.

During the merge, the indexer records the byte offset of each token in `index_seek.json`. It also writes `doc_map.json`, which maps numeric document IDs to URLs, and `analytics.json`, which summarizes the index.

3 Search and Ranking

3.1 Disk-Based Retrieval

The searcher loads the compact seek table and document-to-URL map, but it does not load the full inverted index into memory. For each stemmed query term, it uses the term’s byte offset to seek directly to the corresponding postings line in `index.txt`.

3.2 TF-IDF Ranked Retrieval

Candidate documents are the union of the retrieved postings lists. For each query term t and document d , the base contribution is:

$$\text{score}(t, d) = (1 + \log(tf_{t,d} + 5 \cdot \text{important}_{t,d})) \left(\log \left(\frac{N + 1}{df_t + 1} \right) + 1 \right)$$

when the effective term frequency is positive. Here, N is the number of indexed documents and df_t is the document frequency of the term. Scores are summed across all query terms found in the document.

This is not Boolean AND retrieval: a document matching one query term can be ranked alongside a document matching all terms. Matching more query terms normally increases the accumulated TF-IDF score. A small coverage multiplier also favors documents that match a larger fraction of the query.

3.3 Additional Ranking Components

- Query terms found in a URL receive a bonus.
- Raw text/data URLs and dataset paths receive mild penalties because repeated terms in large data files can otherwise dominate results.
- Equivalent URL forms such as `/`, `/index`, and `/index.html` are collapsed in the final result list.
- URL-based reranking is limited to a preliminary high-scoring window to control query time.

4 Index Analytics

Metric	Value
Indexed documents	55,393
Unique tokens	388,597
Inverted-index size	93,216.91 KB (approximately 91.0 MB)

5 Evaluation

The final searcher was evaluated on 20 queries after the seek table and document map were loaded. Every query completed within the developer-track 300 ms target.

Query	Time (ms)
cristina lopes	7.7
machine learning	87.8
ACM	92.7
student affairs	105.3
artificial intelligence	67.2
data mining	88.0
donald bren school	72.6
secure systems	107.8
software engineering masters	131.1
informatics graduate program	93.1
master of software engineering	185.1
database systems	115.1
computer vision	77.2
mondego lab	78.6
undergraduate admissions	80.6
ics faculty	102.0
course catalogue	102.1
networked systems	78.1
game design	87.7
uci calendar	128.0
Average	94.4
Maximum	185.1
Queries under 300 ms	20 of 20

The move from Boolean AND to union-based TF-IDF retrieval improved recall: pages may now be returned even when they contain only part of a multi-term query. The tested results were generally relevant, and URL-aware ranking helped surface program, calendar, faculty, and course pages. Some broad queries still rank publication pages or dataset pages above the most authoritative page, showing that TF-IDF alone does not fully model page quality.

6 Operational Requirements

- The index is stored in filesystem files rather than a database.
- The indexer offloads its in-memory index to disk repeatedly and merges the partial indexes at the end.
- The searcher performs direct disk seeks and does not load all postings into memory.
- The search interface is an interactive console prompt and identifies results by URL.
- The measured response time remained below 300 ms for all 20 evaluation queries after startup.

7 Limitations and Future Improvements

- Ranking quality could be improved with document-length normalization such as cosine normalization or BM25.
- Content-based exact and near-duplicate detection would reduce duplicate pages beyond the current URL normalization.
- Positional indexes and phrase matching would improve multi-word queries.
- PageRank, anchor text, or domain-quality signals could help authoritative pages outrank repeated-term publication and dataset pages.

8 Running the System

```
python3 indexer.py developer.zip -o output
python3 searcher.py --index-dir output
python3 searcher.py --index-dir output --query "machine learning" --top 5
```